Grafana k6 Performance Engineering Masterclass

Official Study Guide & CI/CD SLA Reference | Batch 15 (Road to SDET)

Document Overview: *This publication compiles 100% of the instruction from the 2-hour k6 masterclass by mentor Shouvik Datta. Includes cross-platform installation, test execution CLI, the AI Prompting Framework for writing k6 scripts, all 7 load testing variants, full production scripts, and SLA threshold quality gates.*

1. Installation & Execution Guide (All Platforms)

k6 is distributed as a single compiled Go binary. It does not require Node.js, Python, or Java to execute.

Operating System	Installation Command
Windows (WinGet)	winget install k6 --source winget
Windows (Choco)	choco install k6
macOS (Homebrew)	brew install k6
Linux (Ubuntu/Debian)	sudo apt-get install k6
Docker (Zero Install)	docker run --rm -i grafana/k6 run - <script.js

Essential CLI Execution Commands:

```
# Basic test run:
k6 run script.js

# Override concurrency and duration on the fly:
k6 run --vus 25 --duration 2m script.js

# Export summary report to JSON or CSV:
k6 run --summary-export=summary.json script.js
k6 run --out csv=results.csv script.js

# Pass dynamic environment variables:
k6 run -e BASE_URL="https://api.mycompany.com" -e USER_COUNT=50 script.js
```

2. How to Prompt AI to Write High-Quality k6 Scripts

When asking AI (ChatGPT, Claude, Gemini) to generate k6 scripts, you must supply strict architectural constraints to prevent the model from hallucinating unsupported Node.js packages.

The Golden Prompt Framework:

- **Role:** Expert Performance Engineer using Grafana k6 with modern ES6 syntax.
- **Runtime Constraint:** Native k6 modules only (k6/http, k6). No Node.js modules (axios, fs).
- **Lifecycle Architecture:** Perform auth once in setup() and pass token to default(data).
- **Data Pacing:** Explicit think time (sleep(1)) and JSON serialization.
- **SLA Quality Gates:** Strict thresholds for latency (p95 < 800ms) and error rate (< 1%).

Copy-Paste AI Mega-Prompt:

```
You are an expert Performance Engineer writing tests with Grafana k6.
Please write a complete, production-grade k6 test script with the following requirements:

1. Target API:
   - Auth Endpoint: POST https://api.example.com/v1/auth
     Payload: { "username": "test_user", "password": "SecurePassword123" }
     Expected response: { "token": "<jwt_token>" }
   - Main Endpoint: POST https://api.example.com/v1/orders
     Headers: Authorization: Bearer <token>, Content-Type: application/json
     Payload: { "productId": 402, "quantity": 1, "paymentMethod": "CARD" }

2. Architecture:
   - Authenticate ONCE in export function setup() and return the token.
   - export default function(data) must use data.token for the main request.
   - Use JSON.stringify() for payloads and res.json() for response parsing.
   - Include realistic think time (sleep 1-2s). Native k6 modules only.

3. Traffic Profile:
   - Ramp-up: 0 to 20 VUs in 30s | Steady: 20 VUs for 2m | Ramp-down: to 0 in 20s.

4. Assertions & SLAs:
   - Check status 200, response time < 600ms, and valid order ID.
   - Thresholds: http_req_duration ['p(95)<800'], http_req_failed ['rate<0.01'].
```

3. The 7 Performance Testing Variants in k6

Performance testing comprises multiple distinct traffic profiles depending on test objectives:

Variant	Objective	Stages Configuration
1. Smoke Test	Minimal sanity check (1 VU) to verify script logic.	stages: [{ duration: '1m', target: 1 }]
2. Load Test	Validates standard expected peak daily traffic.	stages: [{ duration: '5m', target: 50 }, { duration: '20m', target: 50 }, { duration: '3m', target: 0 }]
3. Stress Test	Pushes beyond normal limits to find breaking points.	stages: [{ duration: '5m', target: 100 }, { duration: '10m', target: 250 }, { duration: '5m', target: 400 }]
4. Spike Test	Sudden massive burst (flash sales) to test elasticity.	stages: [{ duration: '10s', target: 10 }, { duration: '15s', target: 500 }, { duration: '1m', target: 500 }]
5. Soak Test	Continuous load for 2-12 hours to catch memory leaks.	stages: [{ duration: '10m', target: 30 }, { duration: '4h', target: 30 }, { duration: '10m', target: 0 }]
6. Breakpoint	Steps up load indefinitely until the system crashes.	stages: [{ duration: '10m', target: 200 }, { duration: '10m', target: 600 }, ...]
7. Capacity	Finds max throughput within strict SLA limits.	thresholds: { http_req_duration: ['p(95)<500'] }

4. Production k6 Scripts Extracted from Class

Script A: Minimal Baseline Request (script.js)

```
import http from 'k6/http';
import { sleep } from 'k6';

export const options = {
    vus: 1,
    iterations: 1,
};

export default function () {
    http.get('https://quickpizza.grafana.com');
    sleep(1);
}
```

Script B: Decoupled Auth Architecture (script3.js)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
    vus: 5,
    duration: '20s',
};

// 1. SETUP: Authenticates ONCE globally
export function setup() {
    const authPayload = JSON.stringify({ username: "admin", password: "password123" });
    const authRes = http.post("https://restful-booker.herokuapp.com/auth", authPayload, {
        headers: { "Content-Type": "application/json" }
    });
    check(authRes, { "auth status 200": (r) => r.status === 200 });
    return { token: authRes.json().token };
}

// 2. VU EXECUTION: Reuses the shared session token
export default function (data) {
    const bookingPayload = JSON.stringify({
        firstname: "John", lastname: "Doe", totalprice: 150, depositpaid: true,
        bookingdates: { checkin: "2024-01-01", checkout: "2024-01-05" }, additionalneeds: "Dinner"
    });
    const res = http.post("https://restful-booker.herokuapp.com/booking", bookingPayload, {
        headers: { "Content-Type": "application/json", "Cookie": `token=${data.token}` }
    });
    check(res, { "booking created": (r) => r.status === 200, "latency < 1s": (r) => r.timings.duration < 100
    sleep(1);
}
```

Script C: Production CI/CD SLAs with Thresholds (script6.js)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
    stages: [
        { duration: '5s', target: 5 },   // Ramp up
        { duration: '20s', target: 15 }, // Peak load
        { duration: '10s', target: 0 },  // Cool down
    ],
    // Programmatic Service Level Objectives (SLOs)
    thresholds: {
        http_req_duration: ['p(95)<1000'], // 95% of requests must complete under 1s
        http_req_failed: ['rate<0.01'],    // Errors must remain below 1%
        checks: ['rate>0.99'],             // 99% of checks must pass
    },
};

export default function () {
    const res = http.get('https://grafana.com/docs/k6/latest/');
    check(res, { 'status is 200': (r) => r.status === 200 });
    sleep(1);
}
```

5. Concurrency Math & Metric Interpretation

• **p95 / p90 Percentile vs Average:** Average is susceptible to outliers. p95 guarantees 95% of requests were strictly at or below that latency.
• **Shared Iterations:** vus: 10, iterations: 30 provisions a shared work queue where fast users take more iterations.
• **Throughput Formula:** Throughput (RPS) = Active VUs / (Latency + Think Time).
• **Checks vs Thresholds:** Checks are non-blocking telemetry. Thresholds are pass/fail gates that exit with code 99 to fail CI/CD builds.